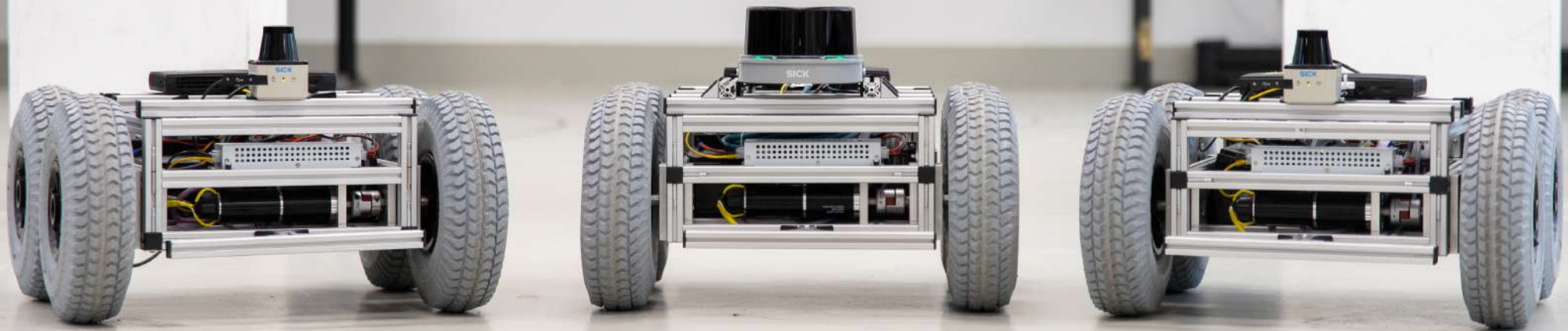


Robotik

Prof. Dr. Thomas Wiemann - FB AI - Robotik



Hochschule Fulda
University of Applied Sciences





Gliederung

Sensordatenverarbeitung

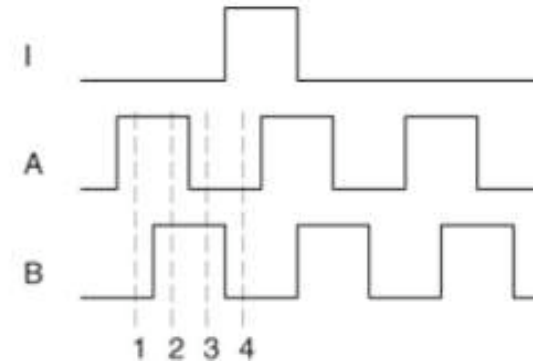
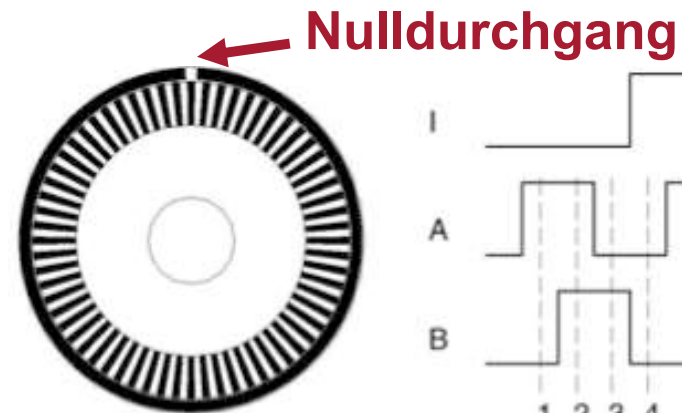
1. Allgemeines
2. Globale Positionsbestimmung
3. Bewegungsmessung /
Odometrie
4. Entfernungsmessung
5. Kameras / Bildverarbeitung
8. Ausblick



- ▶ Grundidee: Mit den Achsen rotieren Messscheiben, die optisch ausgelesen werden.
- ▶ Typische Auflösung (“**Counts per Revolution**”, CPR):
 - ▶ Mobile Robotik ~2000 CPR
 - ▶ Industrieroboter ~10.000 CPR (counts per revolution).
- ▶ Bei bekannter Übersetzung: Anzahl Umdrehungen x Radumfang = Wegstrecke
- ▶ **Odometrie**



Viele Ablesepunkte
phasenversetzt
(höhere Auflösung,
Drehrichtung)



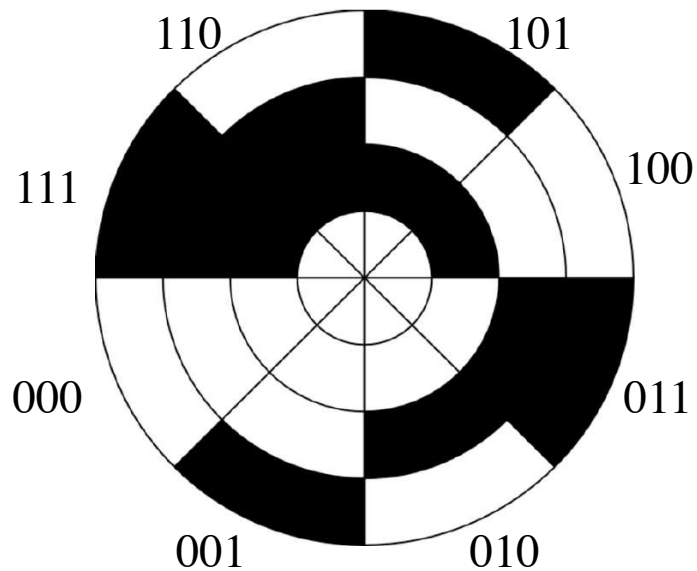
Inkrementalgeber

Nachteil: Absolute Gelenkstellung nicht messbar.

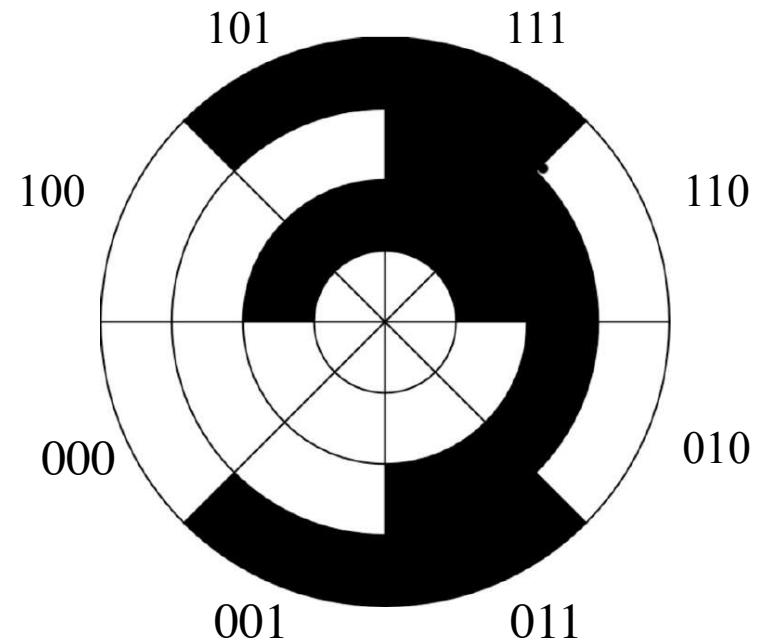
► Beim Einschalten erst „Räkeln“ bis zu einer Absolut-Marke



- Erlauben direktes Ablesen des Gelenkwinkels (auch bei Stillstand)



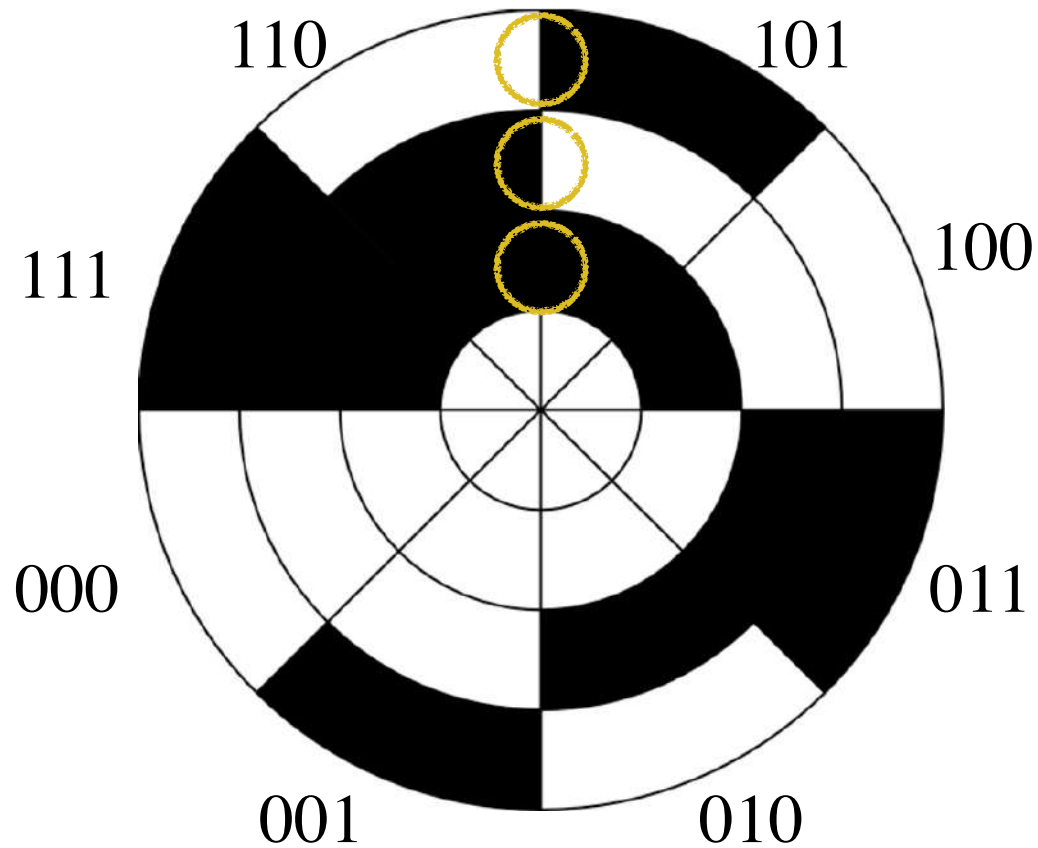
Binärer Absolutgeber (3 Bit)



Graycode-Absolutgeber (3 Bit)
(Robuster gegen Ablesefehler beim
Sektorübergang)



Absolutgeber (2)

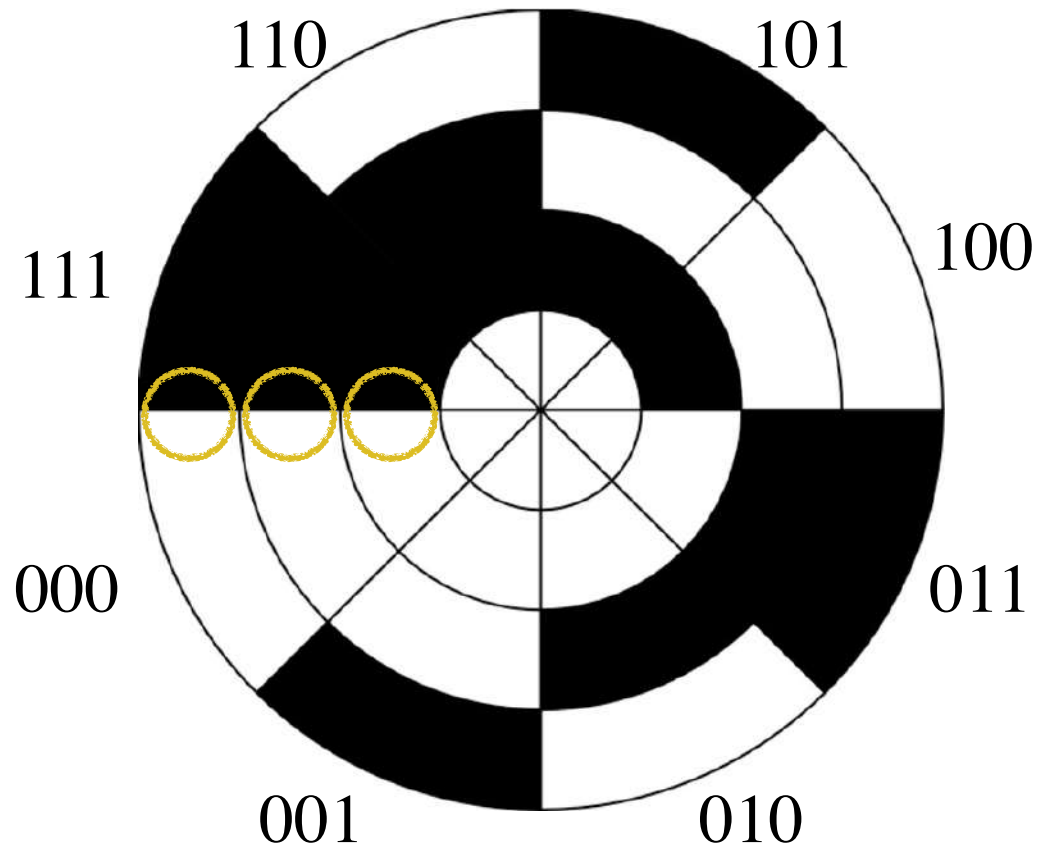


Mögliche Interpretationen:

1 1 0 1 0 1 1 1 1 1 0 0



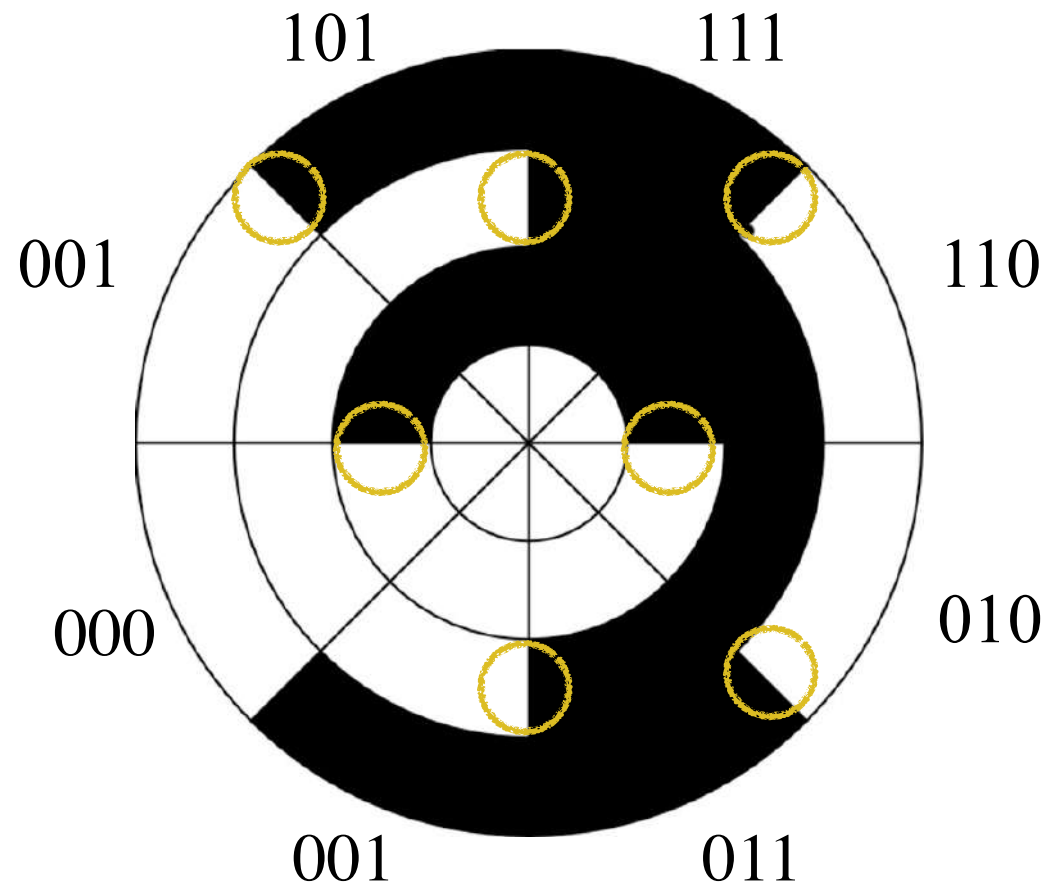
Absolutgeber (2)



Mögliche Interpretationen???



Absolutgeber (3)

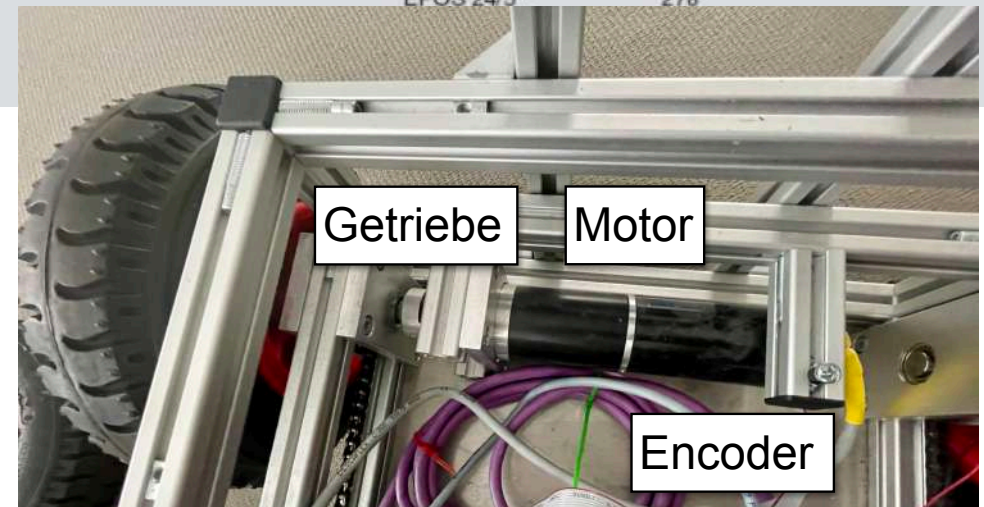
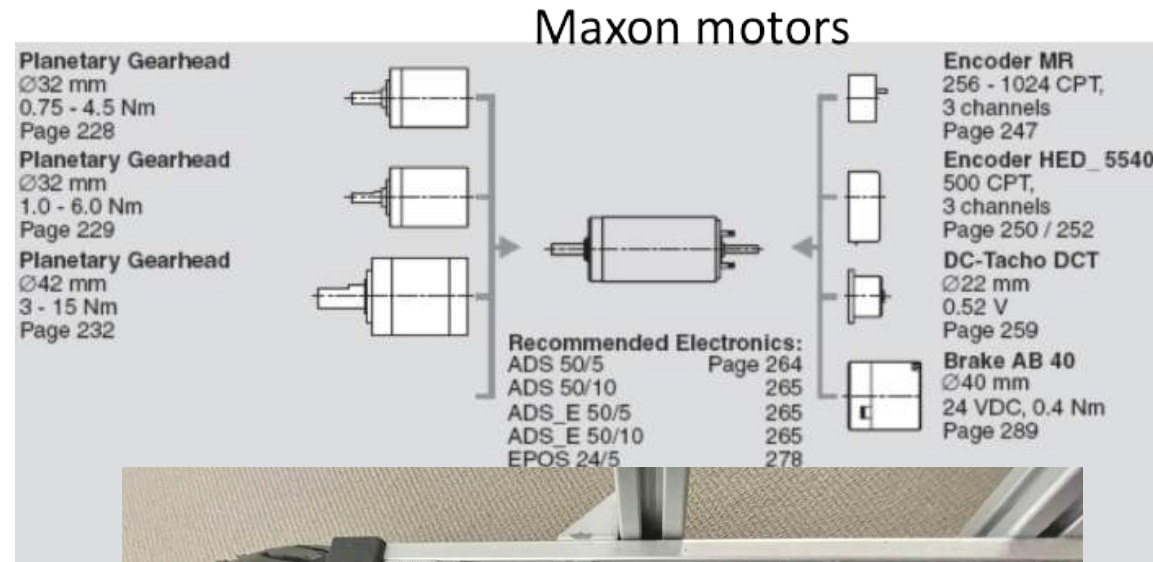


Graycode: Pro Sektorübergang
nur 1 Mehrdeutigkeit!



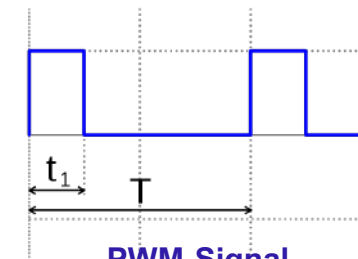
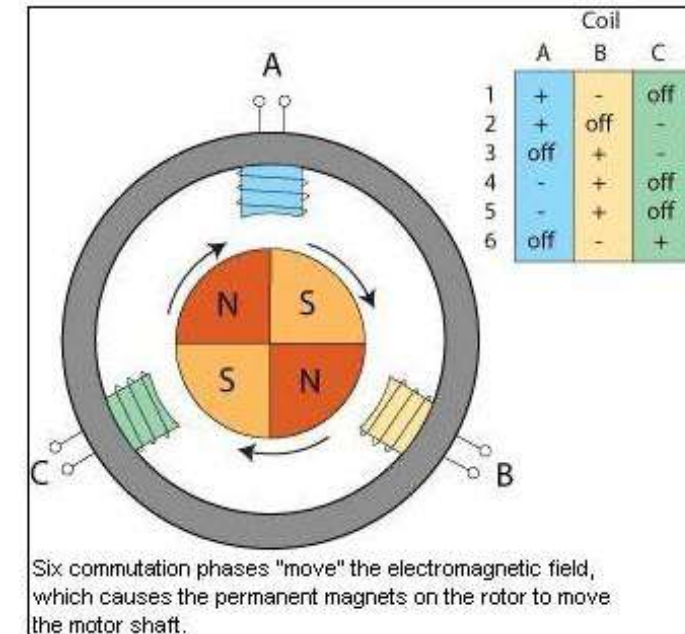
Encoder beim Volksbot

- ▶ Inkrementalgeber sind direkt am Motor angebracht
- ▶ Vorteile:
 - Einfach und robust
 - Eine Baueinheit
 - Günstig
- ▶ Nachteile:
 - Getriebe zwischen Motor und Rädern erzeugen Ungenauigkeiten
 - Getriebetoleranzen
- ▶ CPR: 5.000 bis 20.000
- ▶ Wie kann man die CPR-Anzahl bestimmen?
 - Empirisch!





- ▶ Für **Gleichstrommotoren**:
 - hohe Spannung am Motor ➔ schnell
 - niedrige Spannung ➔ langsam
 - Wechsel $\oplus, \ominus$ ➔ Drehrichtungswechsel
 - Verbinde $\oplus, \ominus$ (Motor wird Generator) ➔ Bremsen
- ▶ Wie reguliert man Motorspannung?
 - über Widerstand ➔ Verlustleistung (Wärme)
- ▶ **PWM (Pulsweitenmodulation)**
 - schalte Spannung periodisch voll und null
 - dadurch Regelung des Mittelwerts/Integrals pro T
 - feste Auflösung über festem Zeitintervall
 - KURT-Roboter: 0 – 1024, 0 entspricht Vollgas



PWM-Signal

Tastgrad $t_1/T = 25\%$

Wikipedia
„Pulsweitenmodulation“



- ▶ Kinematik beschäftigt sich mit der Geometrie und den zeitabhängigen Aspekten der Bewegung
- ▶ Keine Betrachtung der Kräfte, die die Bewegung verursachen
- ▶ Roboterkinematik: Zusammenhang zwischen der Einstellung der beweglichen Teile des Roboters (Räder, Drehgelenke) und Pose des Roboters
- ▶ **Vorwärtskinematik**: Ableitung der Roboterpose aus den intern einstellbaren Parametern
- ▶ **Rückwärtskinematik (Inverse Kinematik)**: Bestimmung der Parameter aus der Pose des Roboters



Kinematiken

- ▶ In dieser Vorlesung sind mobile Roboter Radfahrzeuge mit starrem Körper
- ▶ Es gibt viele andere Bewegungsformen, sie sind sinnvoll (oft), aber sie kommen hier nicht vor





(Aktiver) Freiheitsgrad

Zahl/Art der Bewegungen, die eine einzelne Komponente (Rad, Gelenk, ...) ausführen kann (translatorische, rotat. DOF). Summe der DOF der Komponenten eines Systems (Roboter).

Effektiver Freiheitsgrad

Dimensionalität der Pose.

Holonomie (informell):

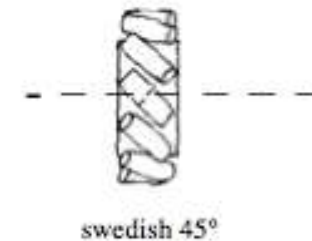
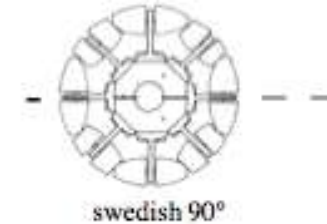
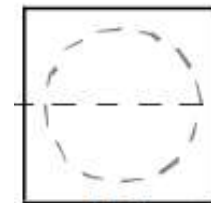
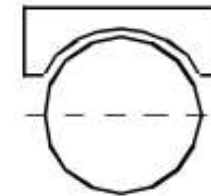
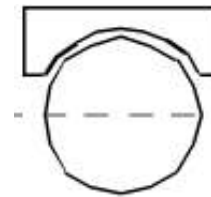
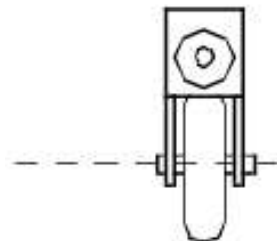
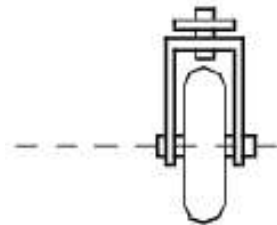
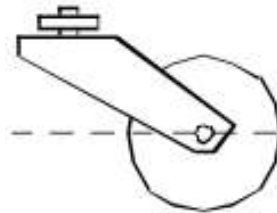
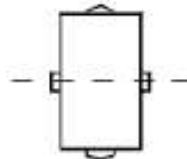
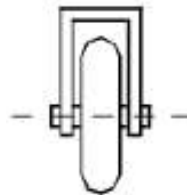
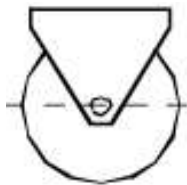
Jeder Wert jeder Dimension des eff. DOF ist unabhängig von den anderen einstellbar durch Änderungen in den aktiven DOF. (System kommt „direkt“ von jeder möglichen Pose in jede mögliche andere.) Ist die Anzahl der effektiven DOF größer als die der DOF, so ist das System nicht-holonom.

► Beispiele

- Starres Rad / Kette: 1 akt. DOF
- Differentialantrieb (Volksbot): 2 aktive DOF
- Volksbot in der Ebene: 3 effektive DOF (3D-Pose (x, y, θ_y))



Räder



Standard-Rad
aktiv, 1-2DOF

Laufrad (*castor wh.*)
passiv, 2 DOF

Sphärisches Rad
meist passiv, 3 DOF

**Mecanum-Rad
(*Swedish*)**
aktiv, 3 DOF

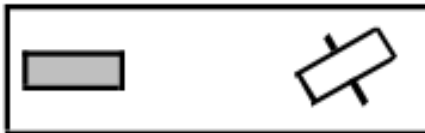


Radkinematiken (1)

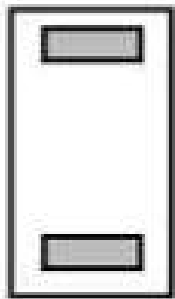
► Zwei- und Dreirad

■ aktives Rad
□ passives Rad

Zweirad



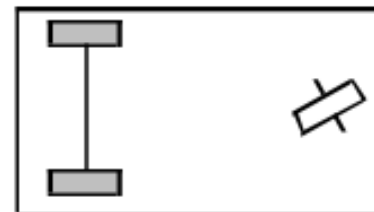
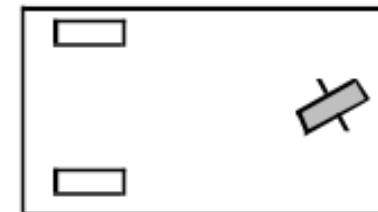
(passiver)
Lenkantrieb



Differenzialantrieb

statisch stabile Kinematik,
falls Schwerpunkt unter
der Achse

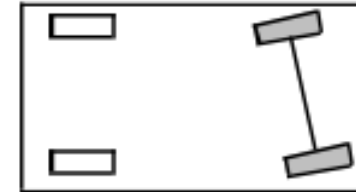
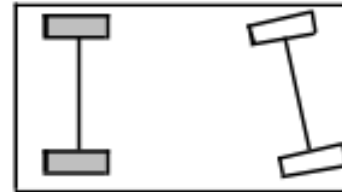
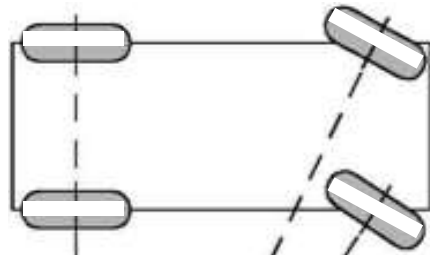
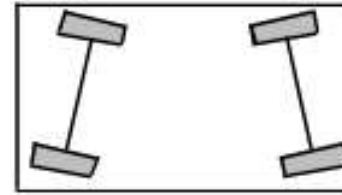
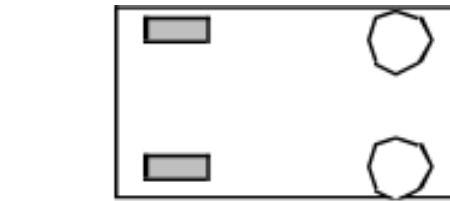
Dreirad (statisch stabil)





Radkinematiken (2)

► Vierräder

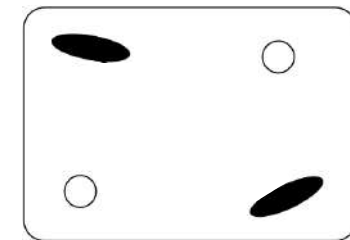


„Ackermann-Kinematik“ (A.-Steuerung)

(Vorder- oder Hinterachse angetrieben)

ICR = *Instantaneous Center of Rotation*
(= ICC, *Inst. Center of Curvature*)

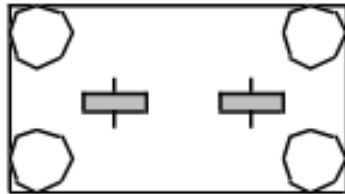
ICR



4 Räder, holonom,
„Fahrradkinematik“



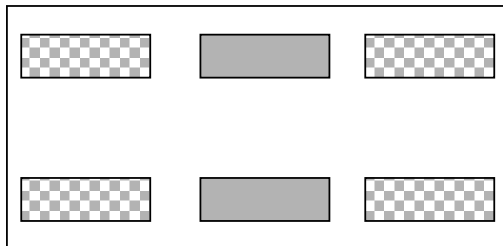
Radkinematiken (4)



KURT2-Kinematik



bzw.

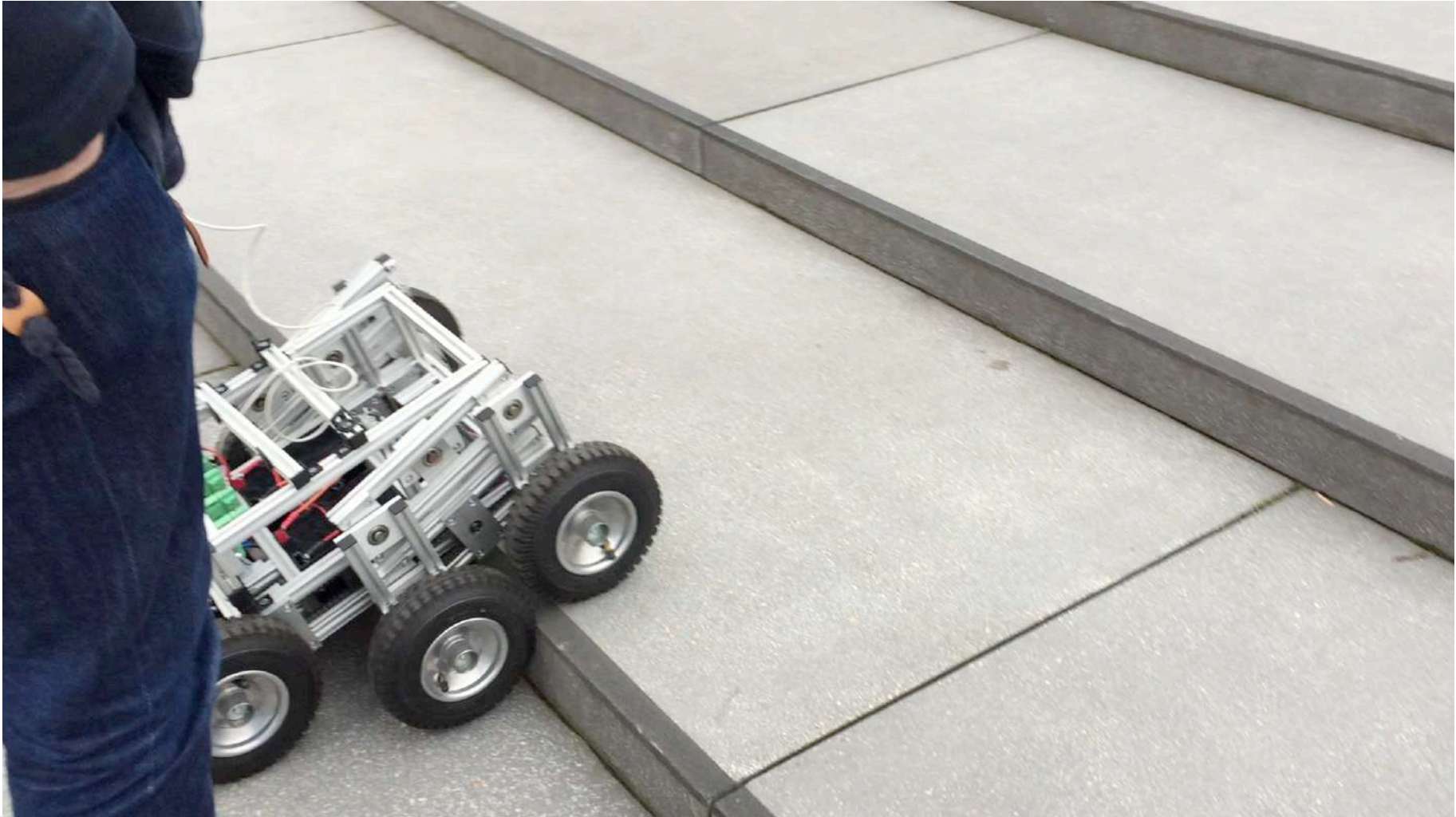


Räder vorn/hinten schleifen!



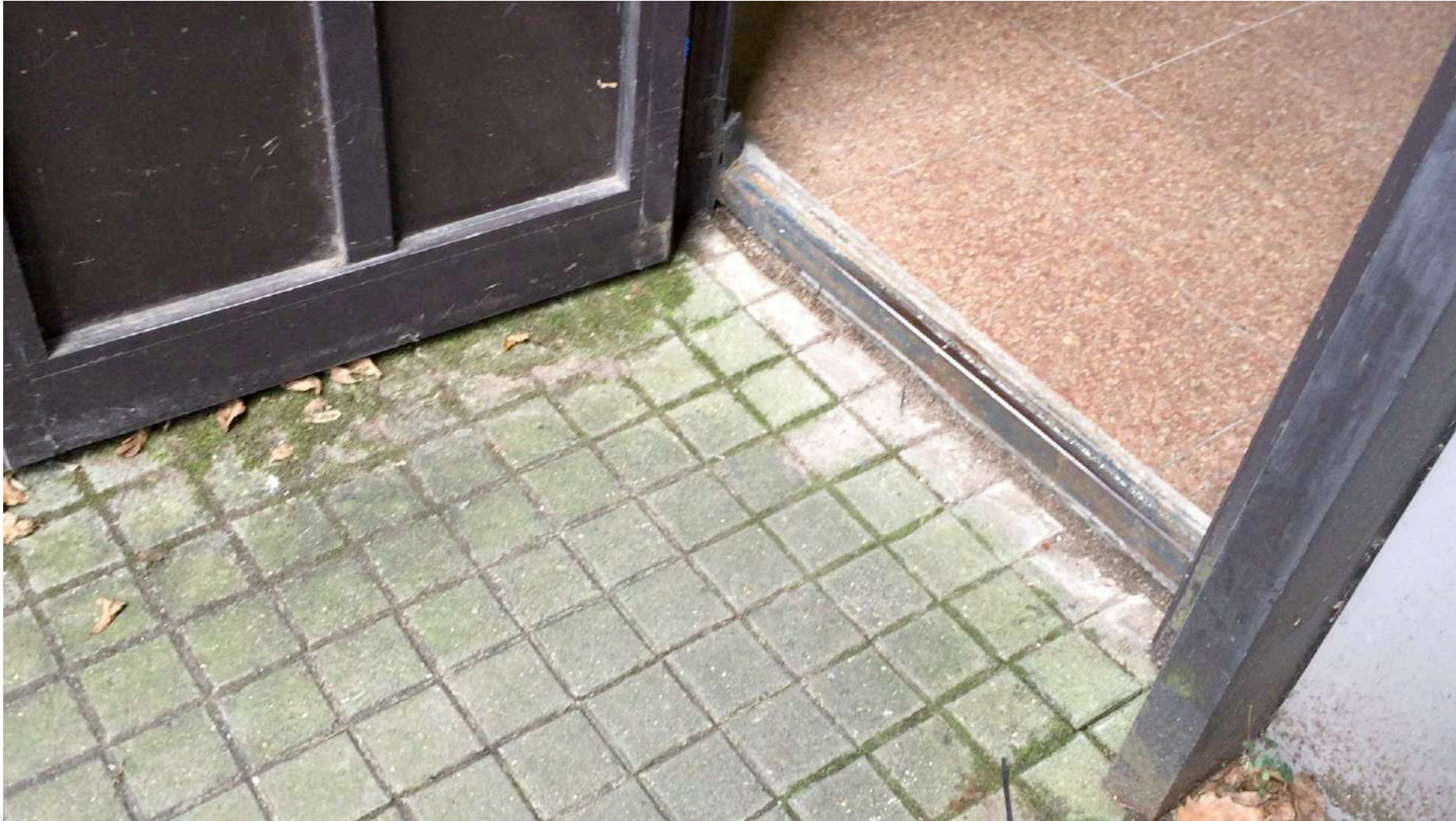


Radkinematiken (6)



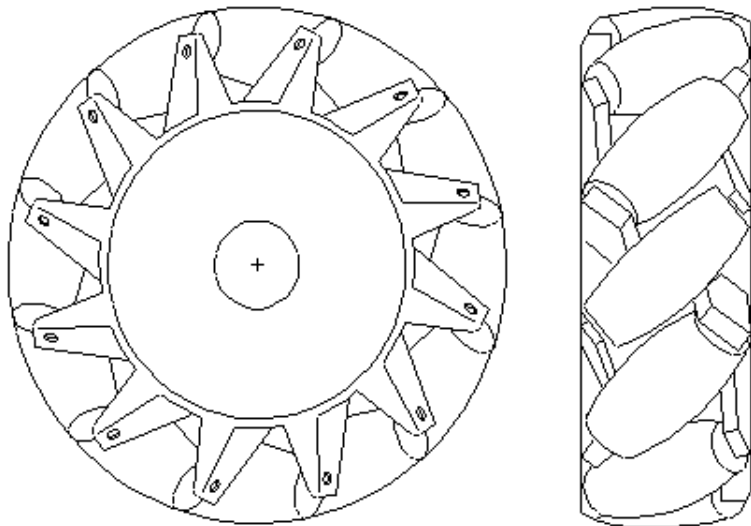


Radkinematiken (7)





Meccanum-Räder



- Gesamträder und Rollen angetrieben
 - Holonom in der Ebene
 - Klappt nur auf planem Boden!





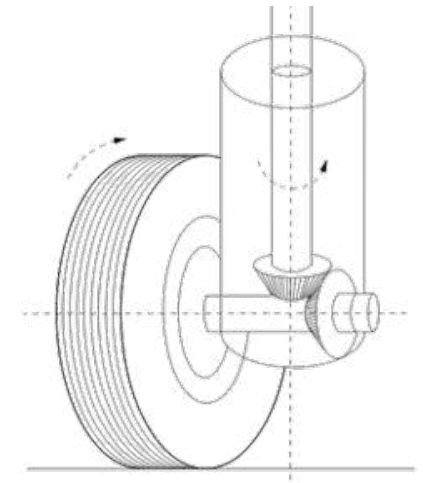
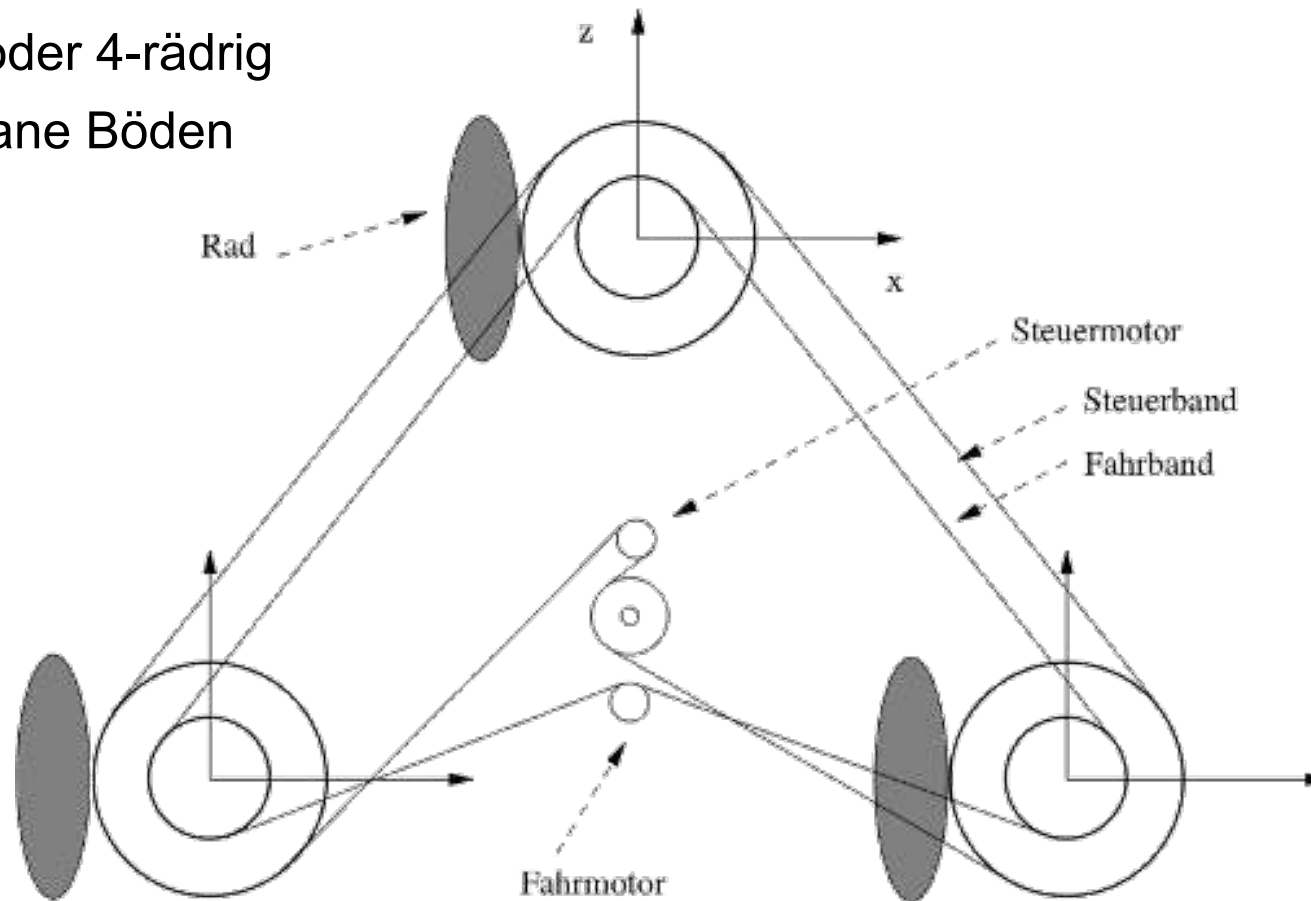
Beispiel: KUKA Youbot





Synchro-Antrieb

- ▶ Holonom
- ▶ Gibt's 3- oder 4-rädrig
- ▶ Nur für plane Böden



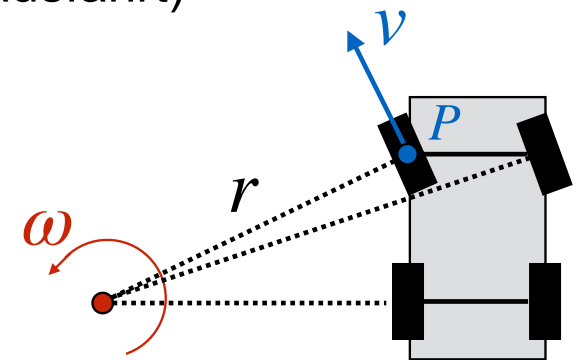


Beispiel





- ▶ Momentanpol (“Instantaneous Center of Rotation, ICR”)
 - ▶ Die Bewegung eines starren Körpers lässt sich zu jedem Zeitpunkt als reine Drehbewegung um einen momentanen Drehpunkt auffassen
- ▶ Rotiert der Körper mit der Winkelgeschwindigkeit ω um den ICR auf einem Kreis mit dem Radius r , dann gilt für seine Geschwindigkeit v in einem Punkt P
$$\omega = \frac{v}{r}$$
- ▶ Geschwindigkeitsvektor v steht dabei senkrecht auf r
- ▶ Der Radius kann unendlich werden. Dann wird $\omega = 0$ (Geradeausfahrt)



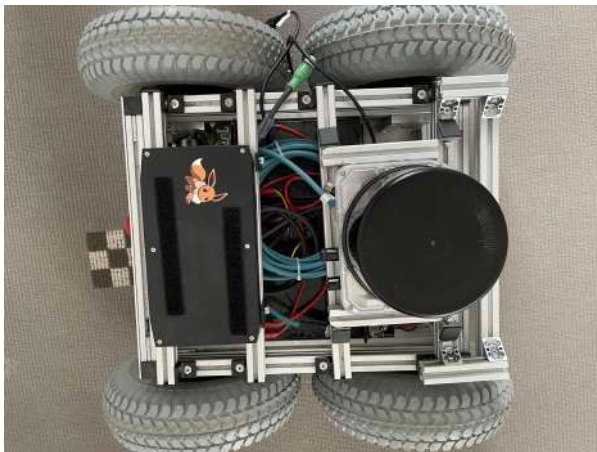


- ▶ **Globale Lokalisierung vs. Inkrementelles Posetracking**
- ▶ Globale Lokalisierung:
 - ▶ Ermittle Pose in externen Bezugssystem (Karte, Koordinatensystem)
 - ▶ Thema des nächsten Kapitels!
- ▶ Inkrementelle Lokalisierung (relative Lokalisierung, lokale Lokalisierung, tracking)
 - ▶ Ermittle Pose bezüglich Startpose bzw. Zwischenpose
 - ▶ Schätze zurückgelegten Weg durch Integration über die gemessene vorwärtskinematische Eigenbewegung
- ▶ Vorteil:
 - ▶ Keine Exterozeption (sondern Vorwärtskinematik)
- ▶ Nachteil:
 - ▶ Vorwärtskinematik approximiert nur die tatsächliche Bewegung

Gesamtfehler der Schätzung durch ist durch die Integration praktisch unbeschränkt!



- ▶ Unterschiedliche Kinematiken erfordern jeweils andere mathematische Modelle!
 - Beispiele (Rad- und Beinantrieb) s. Dudek/Jenkin Kap. 2.1
 - Weitere Kinematiken s. Buch
 - Odometrie bei Beinantrieben unüblich
 - Basis-Odometrie für Rosbot, Volksbot, etc.
 - Exzellente Web-Darstellung: rosum.sourceforge.net/papers/DiffSteer/DiffSteer.html



Koppelnavigation (dead reckoning):

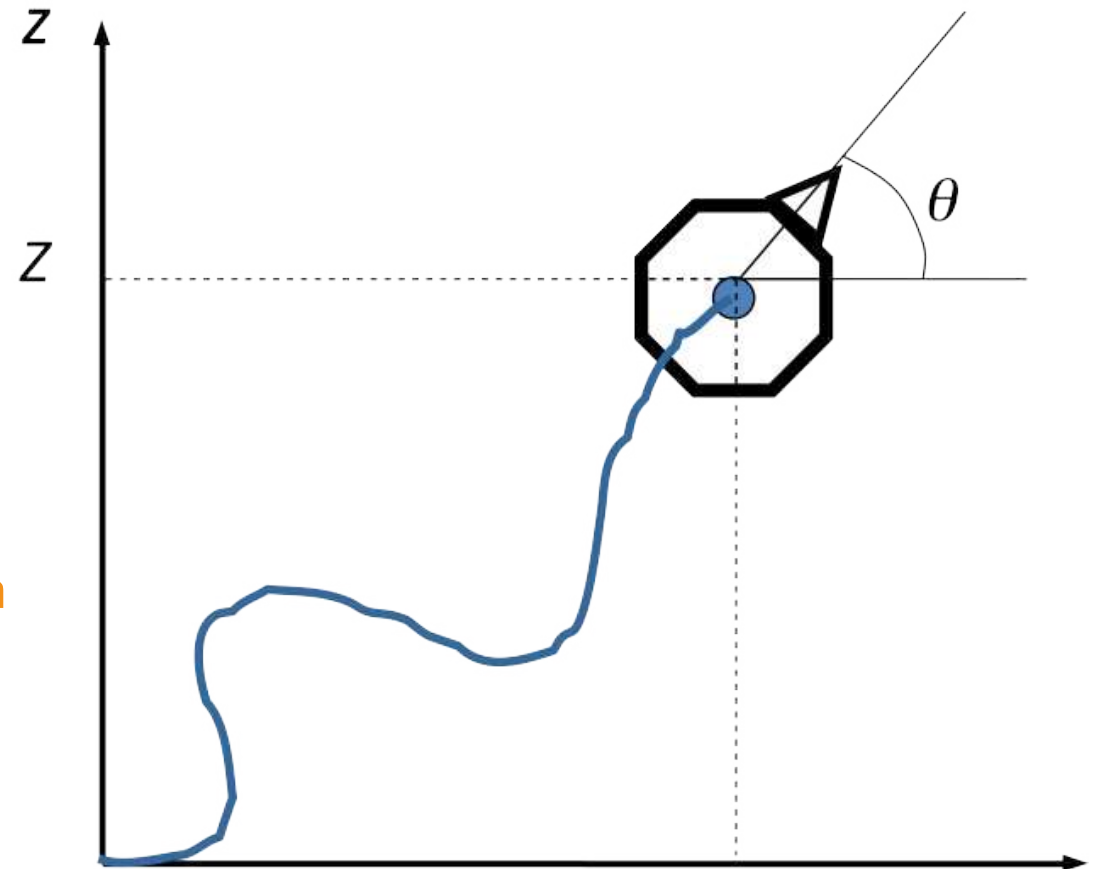
Berechnung d. Trajektorie über fortlaufende Messung von Orientierung, Geschwindigkeit und Zeit



Odometrie für Roboter mit Diff-Antrieb

$$\begin{pmatrix} X \\ Z \\ \theta \end{pmatrix} = \begin{bmatrix} \int dx/dt \\ \int dz/dt \\ \int \theta dt \end{bmatrix}$$

- ▶ Orientierung θ : 0° in x -Richtung, **positiv im Uhrzeigersin**
- ▶ Startpose $[X_0, Z_0, \theta_0]^T = [0, 0, 0]^T$
- ▶ Referenz: Hier Robotermittelpunkt

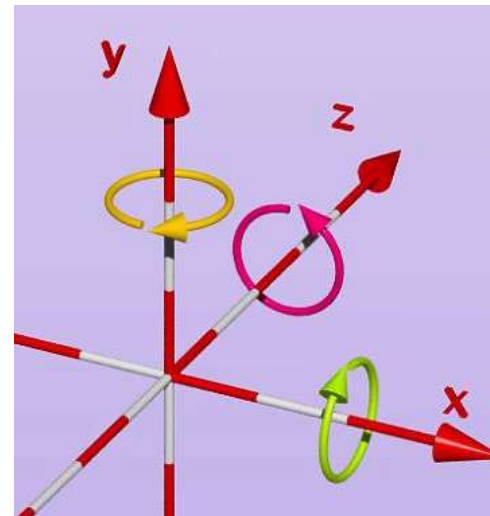




Erinnerung: Koordinatensysteme

- ▶ Erinnerung: Nach Systematik des Skiptes (Buch) rechnen wir in einem **linkshändigen** Koordinatensystem
- ▶ Auch in Computergrafik gebräuchlich (Beispiel: POV-Ray)
- ▶ Eselsbrücke für Drehrichtungen im Linkshändigen Koordinatensystem
- ▶ Diese Drehung um y ist es, die wir in der Ebene betrachten!

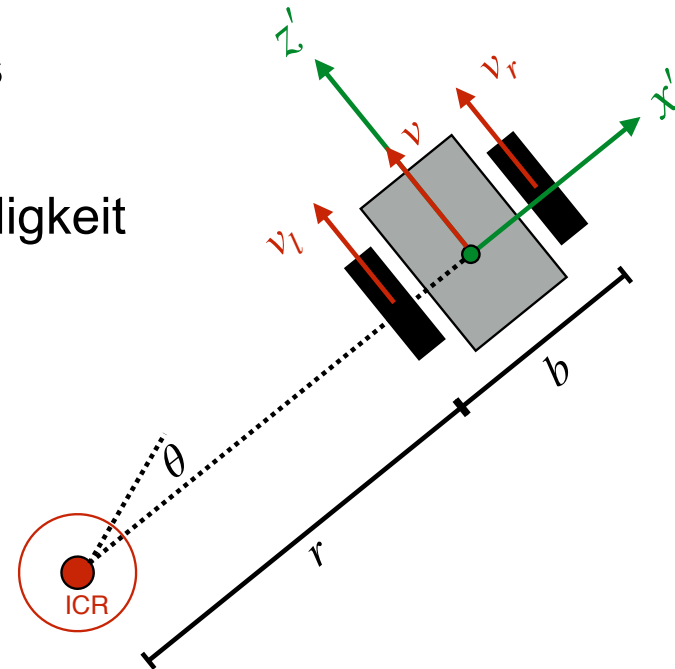
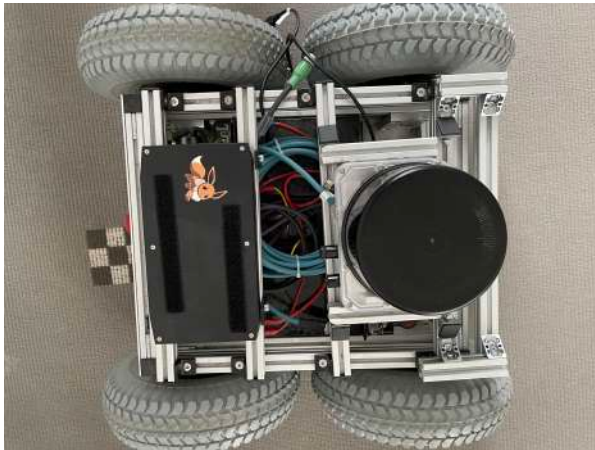
Die positive Drehrichtung um y (und **nur um y**) ist hier im Uhrzeigersinn!





Differenzialantrieb (1)

- ▶ Kinematisches Modell: Roboter wird von zwei unabhängigen Rädern angetrieben
- ▶ Geschwindigkeit v_l des linken und v_r des rechten Rads werden eingestellt
- ▶ Der Roboter bewegt sich um ICR mit Winkelgeschwindigkeit θ und Geschwindigkeit v in lokaler z -Richtung z'





Differenzialantrieb (2)

- ▶ Es gelten folgende kinematische Zusammenhänge:

- ▶ $v_l = \theta r$

- ▶ $v_r = \theta(r + b)$

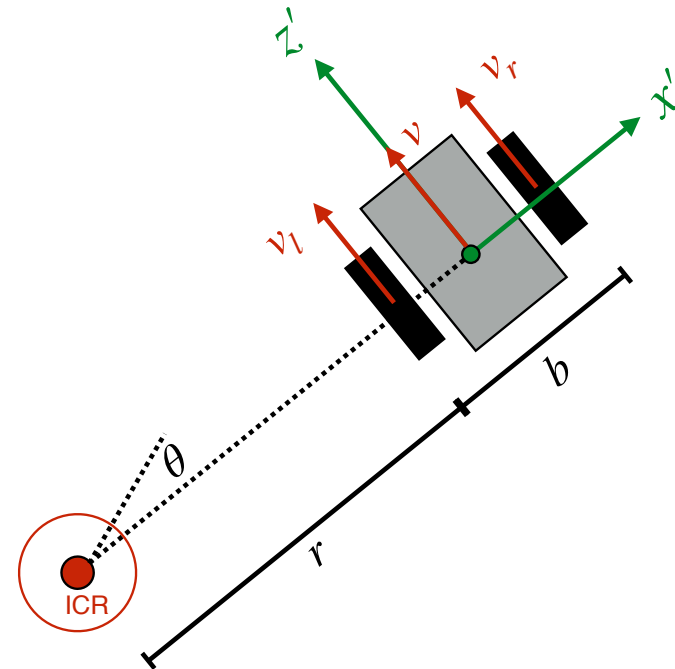
- ▶ $v = \theta(r + b/2)$

- ▶ Daraus ergibt sich:

$$v = \frac{v_r + v_l}{2}$$

$$\theta = \frac{v_l - v_r}{b}$$

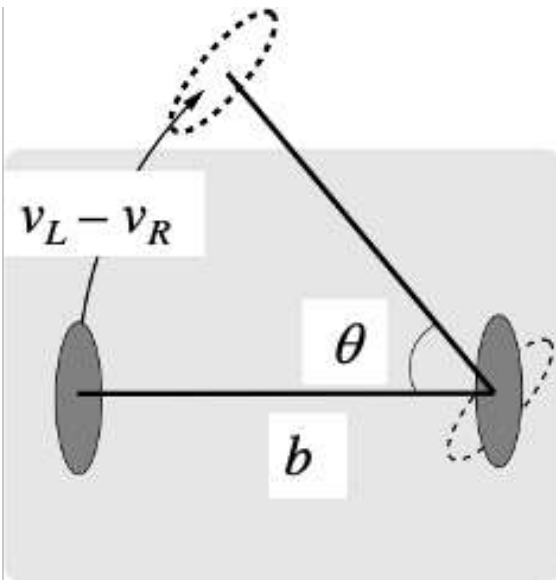
- ▶ Also lassen sich v und θ aus v_l , v_r und der Achslänge b bestimmen
- ▶ Umgekehrt natürlich auch v_l , v_r aus v und θ





Berechnung der Orientierung

- ▶ v_R, v_L : **konstante(!)** Geschwindigkeit des rechten bzw. linken Rads (Projektion. der Nabe)
 - Bekannt aufgrund der Inkrementalgeber / Radumfang
- ▶ b : Achslänge
 - Bekannt durch Konstruktion
- ▶ θ : Schwenkwinkel bei Bewegung
 - hier gemessen als Kreisabschnitt in Radiant rad , $\theta_{\text{rad}} = \theta^\circ \cdot \pi/180$; $1 \text{ rad} \approx 57,3^\circ$



$$\frac{d\theta}{dt} = \frac{v_L - v_R}{b}$$

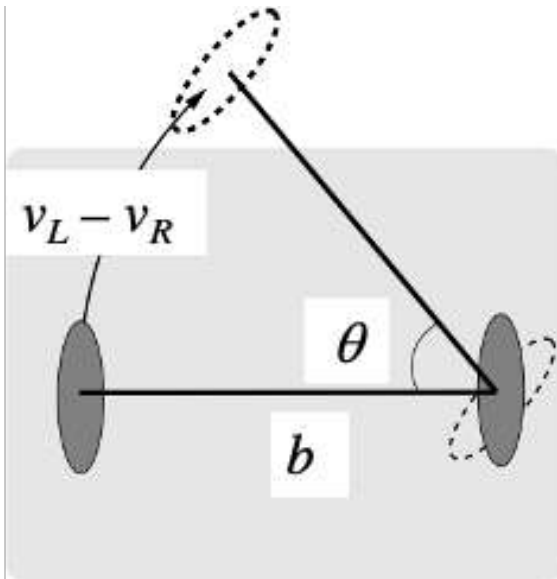
Spezialfälle:

$v_R = v_L$: $d\theta = 0$ (Geradeausfahrt)

$v_L = 0$ oder $v_R = 0$: Segment eines Kreises mit Radius b



- ▶ Integration über t mit Startwert θ_0
- ▶ Berechnung der Pose im “nächsten” Zeitschritt bei Bewegung
- ▶ Konstante Geschwindigkeit (vorausgesetzt)

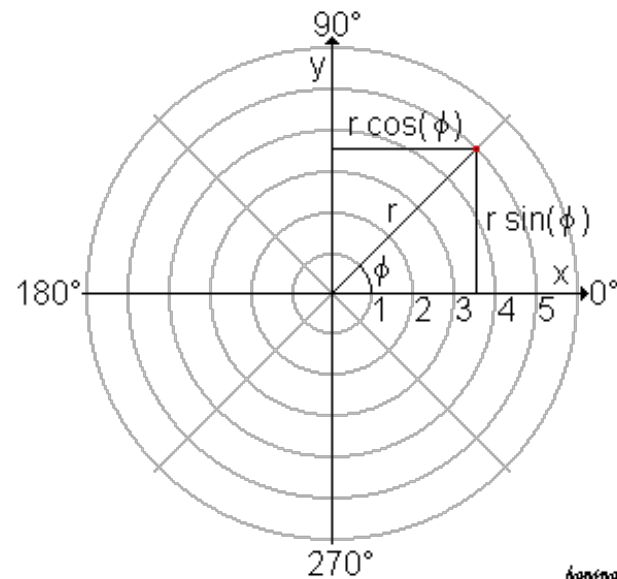
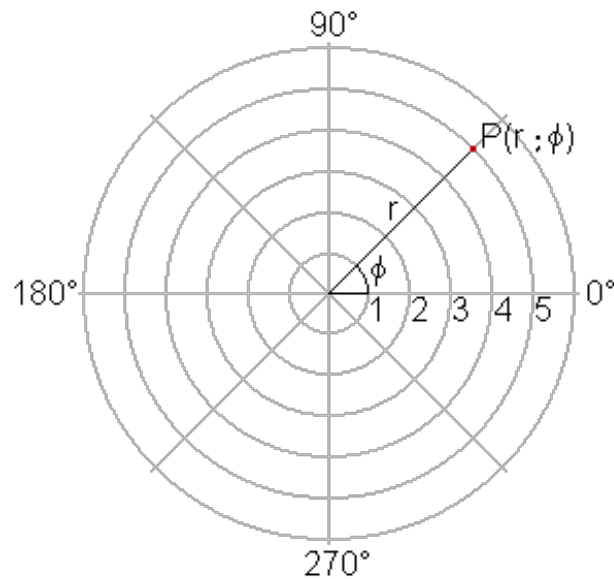


$$\frac{d\theta}{dt} = \frac{v_L - v_R}{b}$$

$$\theta_t = \theta_0 + \frac{(v_L - v_R)t}{b}$$

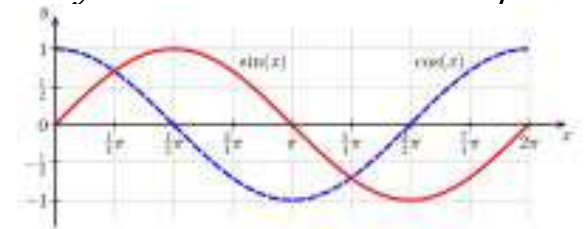


Erinnerung: Umrechnung



$$x = r \cos \varphi$$

$$y \text{ bzw. } z = r \sin \varphi$$

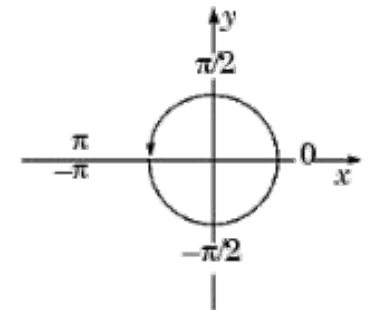


Kartesisch nach Polar:

$$r = \sqrt{x^2 + y^2}$$

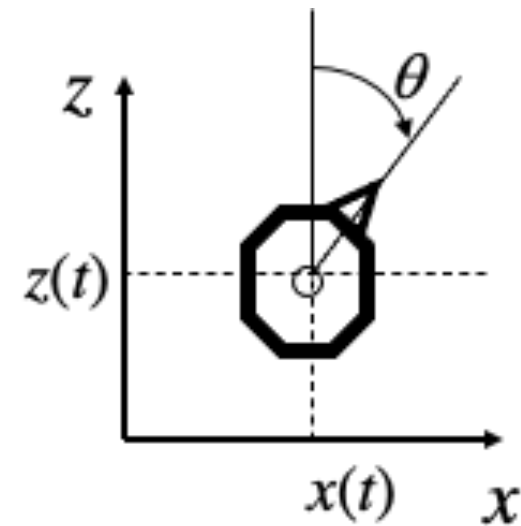
$$\varphi = \arctan \frac{y}{x}$$

atan2 ergibt den Winkel in $[-\pi, \pi]$
des 2-Vektors $(x, y)^T$
(Fallunterscheidung pro Quadrant
über die arctan-Funktion)





Berechnung von x und z



Aufsicht von oben

- ▶ Bei $\theta = 0$ rad: Blick entlang der z -Achse
- ▶ Referenzpunkt im Roboter ist der Achsmittelpunkt
- ▶ Geschwindigkeit des Referenzpunkts: $v = (v_R + v_L)/2$

Spezialfälle:

$v_L = v_R \Rightarrow v = v_L = v_R$ (Geradeausfahrt)

$v_R = 0 \Rightarrow v = v_L/2$ (Kreissegment mit Radius b)

$$dx/dt = v \sin(\theta_t) = [(v_R + v_L)/2] \cdot \sin(\theta_t)$$

$$dz/dt = v \cos(\theta_t) = [(v_R + v_L)/2] \cdot \cos(\theta_t)$$



Berechnung von x und z

- Berechnung der Koordinaten nun analog zur Winkelberechnung
- Integration bei Startwerten x_0, z_0

$$dx/dt = v \sin(\theta_t) = [(v_R + v_L)/2] \cdot \sin(\theta_t)$$

$$dz/dt = v \cos(\theta_t) = [(v_R + v_L)/2] \cdot \cos(\theta_t)$$

$$x(t) = v \int \sin \theta_t dt$$

$$z(t) = v \int \cos \theta_t dt$$

$$\text{mit } \theta_t = \theta_0 + \frac{(v_L - v_R)t}{b}$$

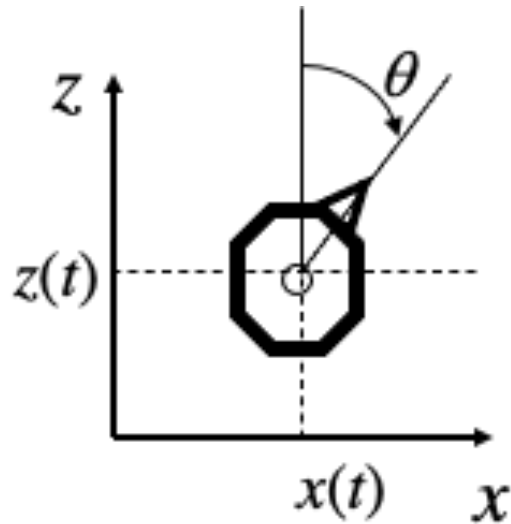
$$x(t) = x_0 + \frac{b (v_r + v_l)}{2 (v_r - v_l)} [\sin(\theta(t)) - \sin(\theta_0)]$$

$$z(t) = z_0 + \frac{b (v_r + v_l)}{2 (v_r - v_l)} [\cos(\theta(t)) - \cos(\theta_0)]$$

Achtung mit Wert $(v_L - v_R)$ im
Nenner der Integrallösung!



Praktische Approximationen



Aufsicht von oben

$$dx/dt = v \sin(\theta_t) = [(v_R + v_L)/2] \cdot \sin(\theta_t)$$

$$dz/dt = v \cos(\theta_t) = [(v_R + v_L)/2] \cdot \cos(\theta_t)$$

$$\theta_t = \theta_0 + \frac{(v_L - v_R)t}{b}$$

- ▶ Die Differenzmessung zwischen v_L , v_R ist fehlerbehaftet
- ▶ Bei kleiner Differenz ($v_L - v_R$) numerisch instabil
- ▶ Approximiere Pose über die Zeit Δt
- ▶ Während Δt seien:

$$v = (v_R + v_L)/2$$

$$\Delta\theta = (v_L - v_R)/b$$

$$\begin{bmatrix} x_n \\ z_n \\ \theta_n \end{bmatrix} = \begin{bmatrix} x_{n-1} \\ z_{n-1} \\ \theta_{n-1} \end{bmatrix} + \begin{bmatrix} v \cdot \Delta t \cdot \sin(\theta_{n-1} + \frac{\Delta\theta}{2} \Delta t) \\ v \cdot \Delta t \cdot \cos(\theta_{n-1} + \frac{\Delta\theta}{2} \Delta t) \\ \Delta\theta \cdot \Delta t \end{bmatrix}$$

- ▶ Alternativen:
 - vereinfacht nur $v \cdot \Delta t \cdot \sin \theta$ bzw. $v \cdot \Delta t \cdot \cos \theta$
 - Kalibrierbares Fehlermodell für x, z (z.B. für Ceres)



Beispiel: Rechnung nach Approximation

$$\begin{bmatrix} x_n \\ z_n \\ \theta_n \end{bmatrix} = \begin{bmatrix} x_{n-1} \\ z_{n-1} \\ \theta_{n-1} \end{bmatrix} + \begin{bmatrix} v \cdot \Delta t \cdot \sin\left(\theta_{n-1} + \frac{\Delta\theta}{2} \Delta t\right) \\ v \cdot \Delta t \cdot \cos\left(\theta_{n-1} + \frac{\Delta\theta}{2} \Delta t\right) \\ \Delta\theta \cdot \Delta t \end{bmatrix} \quad \text{mit} \quad \begin{aligned} v \left[\frac{\text{cm}}{\text{s}} \right] &= (v_R + v_L) / 2 \\ \Delta\theta \left[\frac{1}{\text{s}} \right] &= (v_L - v_R) / b \end{aligned}$$

Seien

- $b = 25\text{cm}$
- $v_L = 80\text{cm/s}$, $v_R = 75\text{cm/s}$
- aktuelle Pose $[0,0,0]^T$
- $\Delta t = 1\text{s}$

$$\Delta\theta \cdot \Delta t = \frac{(80 - 75) \frac{\text{cm}}{\text{s}}}{25\text{cm}} \cdot \Delta t = 0,2 \frac{1}{\text{s}} \cdot \Delta t$$

(Hinweis: $0,2\text{rad} \approx 11,46^\circ$)

$$\begin{bmatrix} x_1 \\ z_1 \\ \theta_1 \end{bmatrix} (1\text{s}) = \begin{bmatrix} 0\text{cm} \\ 0\text{cm} \\ 0 \end{bmatrix} + \begin{bmatrix} 77,5 \sin(0 + 0,1)\text{cm} \\ 77,5 \cos(0 + 0,1)\text{cm} \\ 0,2 \end{bmatrix} = \begin{bmatrix} 7,74\text{cm} \\ 77,11\text{cm} \\ 0,2 \end{bmatrix}$$



Beispiel: Rechnung nach Approximation

- ▶ Geschätzte Pose nach 1s im $[x_0, z_0, \theta_0]^T$ -Referenzsystem: $[7,74\text{cm}, 77,11\text{cm}, 0,2]^T$
- ▶ Die “zweite Sekunde” (ohne Dimensionen der Werte, selbe v_L, v_R)

$$\begin{bmatrix} x_2 \\ z_2 \\ \theta_2 \end{bmatrix} = \begin{bmatrix} 7,74 \\ 77,11 \\ 0,2 \end{bmatrix} + \begin{bmatrix} 77,5 \sin(0,2 + 0,1) \\ 77,5 \cos(0,2 + 0,1) \\ 0,2 \end{bmatrix} = \begin{bmatrix} 7,74 \\ 77,11 \\ 0,2 \end{bmatrix} + \begin{bmatrix} 22,09 \\ 74,04 \\ 0,2 \end{bmatrix} = \begin{bmatrix} 29,83 \\ 151,15 \\ 0,4 \end{bmatrix}$$

- ▶ Geschätzte Pose nach 2s in Referenzsystem: $[29,83\text{cm}, 151,15\text{cm}, 0,4\text{rad}]^T$
- ▶ Geschätzte Pose nach 2s ohne Zwischenpose: $[30,79\text{cm}, 151,91\text{cm}, 0,4\text{rad}]^T$
- ▶ Inkrementiere jeweils letzte Schätzung (relative Lokalisierung)
- ▶ Zykluszeit praktisch viel kürzer als 1s (eher $\sim 1/100\text{s}$)



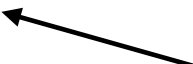
```
geometry_msgs::msg::TransformStamped odom_trans;  
odom_trans.header.frame_id = "odom_combined";  
odom_trans.child_frame_id = "base_footprint";
```

```
odom_trans.header.stamp = now();  
odom_trans.transform.translation.x = x;  
odom_trans.transform.translation.y = y;  
odom_trans.transform.translation.z = 0.0;
```

```
tf2::Quaternion rot;  
rot.setRPY(0, 0, theta);
```

```
odom_trans.transform.rotation.x = rot.x();  
odom_trans.transform.rotation.y = rot.y();  
odom_trans.transform.rotation.z = rot.z();  
odom_trans.transform.rotation.w = rot.w();
```

```
tf_broadcaster_ -> sendTransform(odom_trans);
```

 std::shared_ptr<tf2_ros::TransformBroadcaster> tf_broadcaster_;